01

AULA 01 · Segunda, 10/08

Introdução ao Módulo e Docker

Virtualização, containers e o começo de tudo

Módulo · Arquitetura de Serviços e Dados

Prof.: Everton J Benedicto

ada

Cronograma das Aulas

01
Seg 10/08
Introdução ao Módulo e Docker

02
Qua 12/08
Arquitetura de Servidores

03
Sex 14/08
Apache e Nginx

04
Seg 17/08
Proxy Reverso e DNS

05
Qua 19/08
CI/CD e Ciclo de Vida

06
Sex 21/08
Cloud Computing

07
Seg 24/08
Arquitetura de Dados e Data Warehouse

08
Qua 26/08
Data Lake

09
Sex 28/08
Concluir atividades e preencher rubricas

BOAS-VINDAS

O que vamos aprender neste módulo?

1. **Docker e containers:** empacotar aplicações de forma consistente
2. **Arquitetura de Servidores:** como os sistemas se organizam
3. **Apache, Nginx e Proxy Reverso:** os guardiões do tráfego web
4. **DNS:** o catálogo telefônico da internet
5. **CI/CD e Cloud:** automatizar e escalar com qualidade
6. **Dados:** Arquitetura de Dados, Data Warehouse e Data Lake

💡 ANALOGIA

Pense neste módulo como montar uma pizzaria: cada tema é um ingrediente. No fim, você terá a pizza completa da Arquitetura de Serviços e Dados pronta para servir!

VIRTUALIZAÇÃO

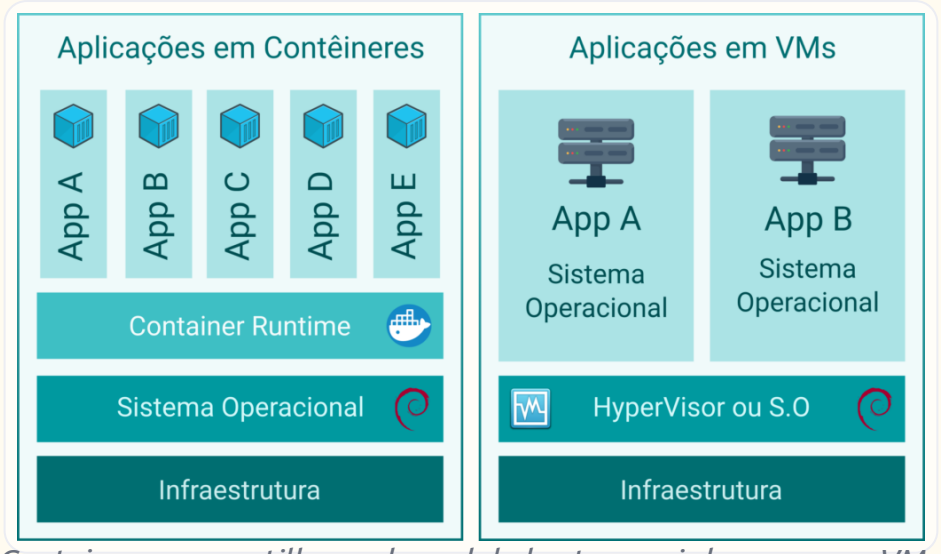
Máquinas Virtuais e o Hipervisor



1. **Virtualização:** vários ambientes isolados em um mesmo hardware
2. **Hipervisor:** camada que divide os recursos entre as VMs
3. **Cada VM:** tem seu próprio Sistema Operacional completo
4. **Benefícios:** aproveitamento do hardware, flexibilidade e isolamento

COMPARAÇÃO

Virtualização vs Conteinerização



Containers compartilham o kernel do host — mais leves que as VMs.

Fonte: 4linux.com.br

DOCKER

O que é o Docker? — Analogia do Condomínio

1. **Terreno (SO Host):** a base onde tudo é construído
2. **Prédio (Docker Engine):** compartilha CPU, memória e rede
3. **Apartamentos (Containers):** cada app isolado em seu espaço
4. **Planta (Dockerfile):** a receita de como construir o container
5. **Catálogo (Docker Hub):** repositório de imagens prontas

💡 ANALOGIA

Apartamentos num condomínio compartilham a estrutura (água, luz, elevador), mas cada um tem cozinha e banheiro próprios. Containers fazem o mesmo: compartilham o SO, mas isolam cada aplicação.

MÃO NA MASSA

Docker na Prática — Comandos Essenciais

Estes são os comandos do dia a dia com Docker:

bash

```
# Verificar a versão instalada
docker --version

# Testar a instalação
docker run hello-world

# Baixar uma imagem e rodar nginx na porta 8080
docker pull nginx
docker run -d -p 8080:80 --name meu-nginx nginx

# Listar, parar e remover
docker ps
docker stop meu-nginx && docker rm meu-nginx
```

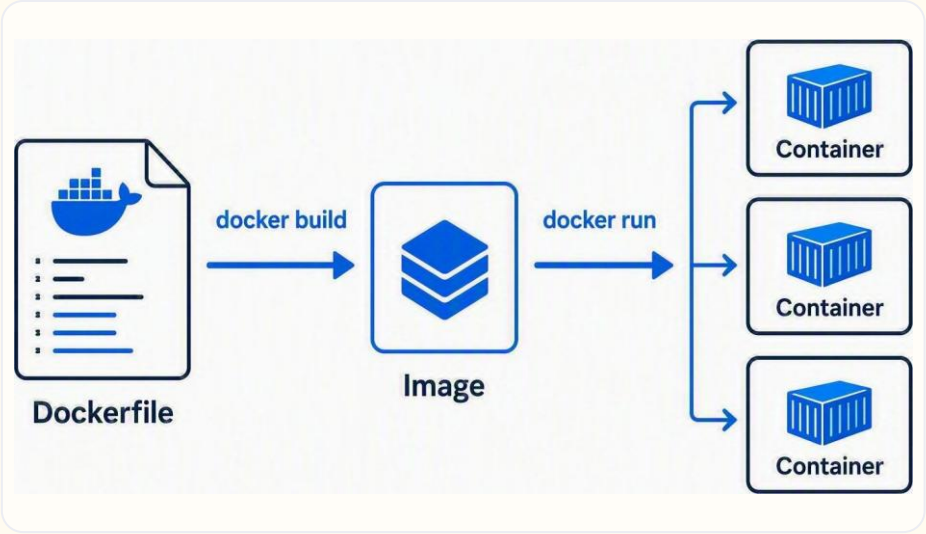
Docker — Comandos Essenciais

bash

```
docker run <nome-da-imagem>
docker run -it <nome da imagem>
docker ps
docker ps -a
docker run -d <nome da imagem>
docker run -d -p porta-local:porta-conteiner <nome da imagem>
docker stop <id ou nome>
docker start <id ou nome>
docker start -it <id ou nome>
docker run –name nome_app <nome da imagem>
docker logs <id ou nome>
docker logs -f <id ou nome>
docker rm <id ou nome>
```

DOCKERFILE

O que é o Dockerfile?



1. **Receita da imagem:** arquivo de texto com as instruções para construir a imagem
2. **Para que serve:** automatiza o build do ambiente com docker build
3. **Por que usar:** o mesmo ambiente roda igual em qualquer máquina
4. **Vantagens:** versionável no Git, reproduzível e fácil de compartilhar

SINTAXE

Principais instruções do Dockerfile

1. **FROM:** define a imagem base a partir da qual você constrói a sua imagem
2. **RUN:** executa comandos durante a construção, como instalar pacotes
3. **COPY:** copia arquivos do host para dentro da imagem
4. **WORKDIR:** define o diretório de trabalho dentro da imagem
5. **EXPOSE:** informa a porta em que a aplicação do contêiner vai escutar
6. **CMD:** comando padrão executado quando o contêiner é iniciado

💡 ANALOGIA

O Dockerfile é a receita de bolo: FROM é a massa pronta, RUN e COPY são os ingredientes que você acrescenta, e o CMD é o momento de servir. Cada instrução vira uma camada da imagem.

Exemplos de Dockerfile — Ubuntu e Nginx

Dockerfile

```
# Exemplo 1 — contêiner Ubuntu
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y curl
WORKDIR /app
COPY . /app
CMD ["bash"]

# Exemplo 2 — servidor web Nginx
FROM nginx:alpine
COPY ./site /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

MÃO NA MASSA

Dockerfile na Prática — do build ao contêiner

Crie um Dockerfile na pasta do projeto e execute:

bash

```
# 1. Criar o arquivo Dockerfile na pasta do projeto
nano Dockerfile

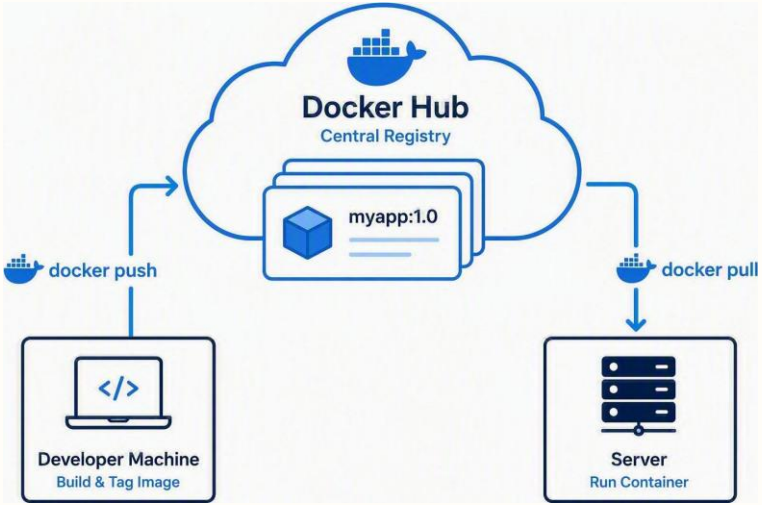
# 2. Construir a imagem (o ponto = diretório atual)
docker build -t meu-site:1.0 .

# 3. Conferir a imagem e subir o contêiner
docker images
docker run -d -p 8080:80 --name site meu-site:1.0

# 4. Testar, ver logs e limpar
curl localhost:8080 && docker logs site
docker stop site && docker rm site
```

DOCKER HUB

O que é o Docker Hub?



1. **Repositório de imagens:** registro público na nuvem onde as imagens Docker ficam
2. **Para que serve:** armazenar, versionar e distribuir imagens com push e pull
3. **Por que usar:** milhares de imagens oficiais prontas: nginx, python, mysql
4. **Vantagens:** compartilhe sua imagem com o time e implante em qualquer servidor

MÃO NA MASSA

Publicando uma imagem no Docker Hub

Crie sua conta no hub.docker.com e execute:

bash

```
# 1. Autenticar na sua conta do Docker Hub
docker login -u seu-usuario

# 2. Marcar a imagem com usuario/repositorio:tag
docker tag meu-site:1.0 seu-usuario/meu-site:1.0

# 3. Enviar a imagem para o Docker Hub
docker push seu-usuario/meu-site:1.0


# 4. Baixar e rodar a imagem em qualquer máquina
docker pull seu-usuario/meu-site:1.0
docker run -d -p 8080:80 seu-usuario/meu-site:1.0
```

QUALIDADE

Boas Práticas com Docker

1. **Imagens oficiais:** do Docker Hub — mais seguras e atualizadas
2. **Imagens mínimas:** prefira Alpine para reduzir tamanho e riscos
3. **Ordene o Dockerfile:** do que muda menos para o que muda mais
4. **Isolamento de rede:** use redes personalizadas entre serviços
5. **Nunca:** armazene senhas ou tokens dentro da imagem

💡 ANALOGIA

É como arrumar a mochila de viagem: na base o que usa por último (dependências fixas), no topo o que abre primeiro (seu código). Assim o Docker aproveita o cache e constrói mais rápido!

RESUMO

Recapitulando a Aula

- Virtualização cria VMs com SO completo; containers compartilham o kernel — mais leves e rápidos.
- Docker empacota a aplicação com todas as dependências, acabando com o 'funciona na minha máquina'.
- Comandos essenciais: pull, run, ps, stop, rm, logs.
- Boas práticas: imagens oficiais, Alpine e nunca segredos na imagem.